Architectural Specification: The Rbotzilla Multi-Agent Financial Orchestration Framework

I. Strategic Overview: Multi-Agent Systems (MAS) for Financial Alpha

The design of "rbotzilla" necessitates a move beyond monolithic algorithmic strategies toward a sophisticated Multi-Agent System (MAS), effectively constructing a digital "hive mind." This approach is mandated by the inherent complexity and heterogeneity of modern electronic markets, which are defined by strategic interactions, uncertainty, informational asymmetry, and intense execution competition.[1]

1.1 The Necessity of Collective Intelligence: Why MAS Outperform Single-Agent Models

A Multi-Agent System functions as a collective intelligence structure where numerous independent agents, each with specialized objectives and decision capabilities, interact constantly with each other and the market environment.[2] This architecture is not merely an engineering choice but a competitive requirement. Research confirms that multi-agent frameworks consistently generate greater returns and stability, demonstrating superior performance compared to traditional, single-agent benchmark strategies across diverse trading timeframes.[2]

This performance advantage stems from the system's inherent robustness against noise. Financial time series data is notoriously noisy, especially at high frequencies. An optimal system employs a hierarchical structure where strategic context and filtered knowledge flow from agents analyzing longer time horizons (macro strategy) down to agents executing in shorter timeframes (micro execution).[2] This temporal filtering is key: by constraining fast execution agents to act only on signals confirmed by higher-level, noise-damped policies, the system avoids over-fitting to fleeting market fluctuations and enhances overall decision stability. Furthermore, by distributing tasks functionally—assigning different agents to roles like Alpha generation, Risk management, and Execution—the system achieves better modularity and maintainability compared to trying to encompass all functions within a single algorithm.

1.2 Defining the Rbotzilla Paradigm: A Hive Mind for High-Stakes Trading

Rbotzilla is conceptualized as a highly adaptable system inspired by collective intelligence models, explicitly utilizing the structure of a "drone AI hive mind" coordinated by a central entity, the "Borg Queen," or Orchestrator.[3] The fundamental goal is to leverage AI-driven strategies and advanced data analysis—including integration of specialized Large Language Models (LLMs)—to optimize trading profitability.[3]

The technological demands of this system dictate a high-performance profile. The focus on GPU acceleration and the investigation of high-bandwidth communication mechanisms, such as NVSwitch Communication between algorithms [3], confirms that Rbotzilla is targeted for high-throughput or ultra-low latency applications where speed is paramount, requiring execution speeds likely below one millisecond.[4] This immediately places constraints on infrastructure, linking the architectural design to specialized High-Frequency Trading (HFT) requirements (see Section V). The central figure, the Orchestrator, is critical. In decentralized agent networks, agents operating only on local information often lead to globally suboptimal or unpredictable market actions.[5] The Orchestrator's designation as the "Borg Queen" signifies its non-negotiable role in maintaining a global market and portfolio state, resolving conflicts (e.g., between a profitability signal and a risk limit), and ensuring stable, coordinated operation across the distributed network.[3]

II. Core Architectural Design: The Rbotzilla Hive Mind Structure

The Rbotzilla architecture must implement a hierarchical, behavior-structured multi-agent framework to model agent heterogeneity and manage strategic complexity.[1] This layered approach explicitly maps the traditional algorithmic trading pipeline into discrete, specialized agents, improving operational transparency and robustness.

2.1 Agent Ontology and Functional Layering

The system requires a strict mapping of functional roles to prevent overlapping responsibilities and ensure clean data flow. The standard algorithmic trading pipeline components—data processing, signal extraction, portfolio management, execution, and evaluation—are formalized as individual agents.[6]

A proven methodology for training such a system is the three-stage process involving adversarial conditions.[1] This process starts by pretraining an **Adversary** agent to actively perturb the environment by adjusting order flow intensity (λ) and mid-price volatility (σ), thereby simulating real market shocks. Next, a **Market Maker** (Agent A, typically a mid-layer agent) is trained using reinforcement learning techniques like Proximal Policy Optimization (PPO) under these simulated adversarial conditions. Finally, specialized **Trading Agents** (Bottom Layer Agents B) are independently trained against the resilient, pre-trained market maker.[1] This training ensures that Rbotzilla's policies are inherently prepared for

periods of high market stress and volatility.

The required specialization of Rbotzilla's components is summarized in Table 1, which outlines the roles necessary for a mission-critical trading operation.

Table 1: Proposed Multi-Agent Roles and Functions in Rbotzilla

Agent Role	**Core Function**	**LLM/ML Technique**	**Primary Output**
Orchestrator (Borg Queen)	Coordinates communication (A2A/MCP) and manages task completion; ensures system stability.	Behavioral Coordination Logic, FIPA-ACL/MCP [6]	Task assignments, Conflict resolutions, Global State Update
Alpha Agent (Sentiment)	Processes news and social data; generates weighted market sentiment signals.	FinGPT/Falcon-7B, Confidence-Weighted Scoring [9]	Sentiment Signal (Weighted, Time-stamped)
Alpha Agent (Technical)	Generates buy/sell signals based on time-series analysis across multiple timeframes.	Hierarchical RL (PPO), Technical Indicators, Time-series Models [1]	Technical Signal (Probability-based, Position Suggestion)
Risk Agent	Monitors system-wide exposure, position size limits, and portfolio volatility; enforces risk policy.	Custom Risk Policy, Real-time P&L Calculations, PPO Constraint Parameter (ω) [1]	Risk Limit Enforcement Signals, Max Drawdown Alerts
Execution Agent	Interfaces directly with the broker API (OANDA, etc.);	Low-Latency API Integration, Execution	Executed Order Confirmation, Slippage Report

	optimizes order placement (e.g., smart order routing).	Algorithms (TWAP/VWAP) [4]	
Audit Agent	Records and validates the entire decision-making chain and execution results for debugging and compliance.	Comprehensive Memory Logging, Transaction Hashes [5]	Audit Log, Performance Metrics [1]

2.2 The Role of the Central Orchestrator (The "Borg Queen" Function)

The Orchestrator, or Assimilink, acts as the central control system, managing all mission-critical decisions.[3] It utilizes the Model Context Protocol (MCP) for control messages between itself and the various specialized agents.[6] This centralized control is essential because, in a decentralized system, unpredictable outcomes often arise when agents operate solely based on localized information, without considering the global state of the network or the portfolio.[5]

The Orchestrator's core function is conflict resolution and error mitigation. For example, when an Alpha Agent generates a strong buy signal, the Orchestrator must ensure the resulting trade adheres to the strict constraints imposed by the Risk Agent, such as maximum exposure or portfolio volatility limits. It acts as a necessary choke point in the decision pathway. Furthermore, the Orchestrator, in collaboration with the Audit Agent, must actively mitigate **error propagation**, a significant systemic risk in multi-agent systems where one faulty decision can rapidly cascade across the network.[5] If, for instance, the Sentiment Alpha Agent begins producing unstable or erroneous signals due to data corruption or model drift, the Orchestrator must detect this divergence (often by comparing it against signals from the Technical Alpha Agent) and immediately pause the component, preventing the faulty input from reaching the execution layer.

2.3 Communication Mechanisms and Protocols

Effective agent-to-agent communication is fundamental for coordinating the thousands of AI trading bots that exchange data and react to market changes.[8] While historical protocols like KQML (Knowledge Query and Manipulation Language) and FIPA-ACL (Foundation for Intelligent Physical Agents – Agent Communication Language) established standards for semantic clarity [8], Rbotzilla requires specialized, high-speed mechanisms.

For high-stakes applications, a dual protocol structure is most effective: the Model Context

Protocol (MCP) handles control messages from the Orchestrator, and a dedicated Agent-to-Agent protocol (A2A) handles communication and collaboration among the specialized agents.[6] Given the necessity for speed, Rbotzilla must employ high-bandwidth mechanisms like NVSwitch Communication [3] and minimize verbose data exchange to avoid network congestion, which can severely impact decision speed and execution fidelity.[8] Crucially, the chosen protocol must ensure **semantic clarity**; all agents must share a consistent ontology—a machine-readable definition of core financial terms (e.g., "market liquidity," "high volatility")—to eliminate misinterpretation and optimize coordination efficiency.

III. Implementing LLM Agents (Open AI's) for Signal Generation

The incorporation of Large Language Models (LLMs) represents a paradigm shift in alpha generation, moving financial analysis from manual processes to automated, real-time insights.[11] These models form the core intelligence of Rbotzilla's Alpha Agents, tasked with integrating unstructured data into actionable trading signals.

3.1 LLMs as Alpha Signal Generators and Data Integration

LLMs facilitate the rapid processing of vast, heterogeneous datasets, combining structured financial metrics with unstructured sources like news feeds, social media sentiment, and earnings call transcripts.[11] To operate effectively in the financial domain, Rbotzilla must utilize **finance-specialized models**, such as derivatives of FinGPT or fine-tuned general models like Falcon-7B, rather than general-purpose LLMs.[9] Domain specialization is essential because financial markets require nuanced understanding of industry jargon and context.[12]

Furthermore, standard LLMs have a finite knowledge cutoff (e.g., one model mentioned was pre-trained up to November 2023 [10]). To ensure the LLM Alpha Agent's reasoning incorporates the latest market events and context, Rbotzilla must implement a Retrieval-Augmented Generation (RAG) framework, such as FinGPT-RAG.[14] RAG optimizes information depth by retrieving external, real-time knowledge, grounding the LLM’s sentiment predictions and mitigating the critical risk of making decisions based on outdated market knowledge.

3.2 Confidence-Weighted Signal Scoring and Signal Integrity

Raw LLM output is inherently probabilistic and cannot be used as a deterministic trading signal. Therefore, Rbotzilla must vet the reliability of the LLM’s output. This is accomplished through a **Confidence-Weighted Scoring** mechanism, which integrates the model's confidence level with the measured sentiment strength. For example, a system might assign 60% weight to the model's confidence and 40% to the sentiment strength to generate a more reliable overall signal.[9]

Crucially, the system must maintain a history of sentiment and price movements to calculate their correlation coefficients in real-time.[9] This dynamic correlation tracking acts as a self-tuning mechanism. If the market is in a regime where positive sentiment is strongly correlated with price increases ($r \approx 0.8$), the Portfolio Agent may increase position sizing suggested by the Alpha Agent. Conversely, if market correlation tracking reveals a weak or inverse relationship (a "sell the news" event), the signal's impact on position sizing must be dynamically dampened or inverted, preventing overexposure during regimes where linguistic cues have temporarily decoupled from price dynamics.[9]

3.3 Mitigating Latency Introduced by LLM Inference

The necessity of real-time sentiment analysis introduces a core architectural challenge: the high inference latency associated with Large Language Models.[13] This lag can become a significant bottleneck in a time-sensitive trading environment.

To overcome this, the LLM pipeline must be designed around an **Event-Driven Architecture** that utilizes asynchronous processing of news data streams.[9] The inference process must be decoupled from the core MARL decision and execution loops. Sentiment signals must be treated as features generated externally and asynchronously, ensuring that LLM latency does not impede the millisecond-sensitive decisions of the Execution Agent. For scalable and efficient operation in production, the resource-intensive LLM agents (such as Falcon-7B fine-tunes [9]) should be deployed using container orchestration solutions like Kubernetes, enabling parallel computation and high throughput for real-time data processing.[13]

IV. MARL Frameworks and Communication Protocols

The selection of a robust Multi-Agent Reinforcement Learning (MARL) framework is essential for defining, orchestrating, and monitoring the complex, collaborative workflows of the Rbotzilla hive mind.

4.1 Selection of the Orchestration Framework

Open-source agentic frameworks such as CrewAI, AutoGen, and LangGraph offer robust foundational capabilities for multi-agent systems.[15] CrewAI and AutoGen are effective for defining collaborating agents that utilize LLMs for sequential, complex tasks, leveraging natural language dialogue and function calling.[16] AutoGen, for instance, allows agents to call Python functions and APIs, and is widely adopted for creating collaborative agent teams.[17]

For a mission-critical financial system, however, the primary concerns are reliability, low latency, and deterministic execution, which require meticulous tracking of state and system performance. LangGraph, with its node-based architecture, is highly suitable for defining stateful agent workflows and offers crucial support for monitoring deployment via tools like LangSmith.[15] While powerful features like dynamic code generation (supported by frameworks

like AutoGen [17]) expand task capability, they introduce unacceptable risks in live execution. Therefore, dynamic code generation must be strictly confined to simulated environments, such as the Backtest Agent's sandbox, and never permitted in the live Execution Agent due to security and stability concerns.

4.2 Managing State and Memory

The complexity of a MARL system necessitates comprehensive state management. Rbotzilla requires a dedicated **Memory Agent** responsible for recording all transient states, prompts, tool calls, and final decisions.[6] This centralized memory serves two primary functions:

1. **Policy Reflection and Improvement:** The complete historical record allows higher-level agents (Planner and Orchestrator) to engage in "reflection," reviewing past actions and comparing outcomes against expectations to improve future policy and address systemic deficiencies.[6]
2. **Audit and Compliance:** The Memory Agent provides an immutable log that forms the basis of the decision chain necessary for compliance and regulatory auditing. In a high-stakes domain, being able to trace the precise logic of every trade executed by the Execution Agent is non-negotiable.[6] This logging is also vital for diagnosing collective failures in a multi-agent system, tracing where non-stationarity or coordination breaks occurred.

V. Model Lifecycle and Infrastructure: From Checkpoint to Production

The transition from a developed policy model to a robust, live trading system requires meticulous attention to infrastructure, specifically MLOps practices and latency mitigation techniques.

5.1 The Significance of AI Checkpoints (frozen2snap.zip)

The file item noted in the initial query, identified as frozen2snap.zip (421.4 MB), strongly suggests the usage of a centralized AI checkpoint.[20] Checkpoints bundle model weights, configurations, and metadata. In the context of MARL, these checkpoints provide several critical benefits:

- **Error and Disaster Recovery:** They protect against hardware failures or unexpected interruptions during long, resource-intensive training cycles, allowing training to resume from the last saved state instead of starting over.[20]
- **Transfer Learning:** A large checkpoint likely represents a pre-trained model (either a foundational LLM or a converged RL policy).[20] Utilizing these checkpoints enables **transfer learning**, significantly reducing the data requirements and training time needed to adapt the model for specific Rbotzilla tasks.[20]
- **Policy Versioning:** Checkpoints are periodically saved to evaluate policy changes and

compare the effectiveness of different specialized agents (e.g., comparing the performance metrics of Agents B_1 versus B_2).[1]

The reliance on these large artifacts mandates a mature MLOps pipeline capable of handling checkpoint storage, version control, and, critically, **integrity management**. Given the high-stakes nature of financial trading, the system must generate and verify cryptographic hashes for the checkpoint file upon transfer or deployment to ensure its integrity is maintained.[20]

5.2 High-Performance Execution Layer and Latency Mitigation

In algorithmic trading, ultra-low latency is synonymous with competitive advantage; a delay of even a millisecond can be the difference between profit and loss.[4] To achieve the necessary speed, the Rbotzilla execution layer must adhere to stringent physical and computational standards.

Co-location is the primary infrastructural method for minimizing latency. This involves physically placing Rbotzilla's servers within or adjacent to the exchange's data center, dramatically reducing the time required for data transmission (network hops) and order execution.[21] This is mandatory for achieving the ultra-low latency necessary for high-frequency or arbitrage strategies.[21]

The Execution Agent itself must be architected as a lean, deterministic core application, potentially written in high-performance languages like Rust or C++ (as favored by specialized HFT backtesting frameworks [22]), focused solely on speed and API interaction. This layer must be structurally separated from the slower, reasoning-intensive MARL and LLM components to ensure that complex agent decision-making does not introduce crippling execution delays.

5.3 Deployment Strategies: Scalability and Reliability

For scalable and efficient operation, especially concerning the compute demands of LLM inference, the entire Rbotzilla ecosystem should be deployed using container orchestration platforms like Kubernetes.[13] This strategy ensures that computational resources are allocated appropriately across the heterogeneous agents and facilitates reliability. The deployment mechanism must prioritize fault tolerance, relying on the robust checkpointing and memory systems to quickly recover or restart specialized agents if hardware failures or unexpected crashes occur.[5]

VI. Addressing Non-Stationarity and Systemic Risk

The environment of the stock market is arguably the most extreme example of a non-stationary system.[23] Since the actions of all agents modify the environment, the underlying assumptions of the Markov property, essential to most classical Reinforcement

Learning (RL) algorithms, are violated.[24] This non-stationarity is the single greatest threat to sustained alpha generation, resulting in quick policy degradation through **concept drift** and the loss of accumulated knowledge through **catastrophic forgetting**.[23]

6.1 Solutions for Non-Stationary Environments

Rbotzilla must be designed based on the premise that the market will inevitably change, necessitating continuous adaptation techniques.[23] A purely offline or purely online training regime is insufficient. A powerful solution involves training an **Ensemble of Experts**.[23]

This strategy requires training a diverse set of specialized policy models offline, with each model being an "expert" in a specific market regime (e.g., bull, bear, or volatile regimes). An online learning algorithm then manages the blending or selection of these experts in real-time, allowing the system to rapidly adjust its behavior as new market data arrives.[23] This approach is an effective alternative to continuous full retraining and is often informed by **Meta-Learning** techniques, which train the system on *how to adapt* quickly to regime changes, minimizing the regret bounds typical of non-stationary environments.[25] Continuous adaptation must be paired with real-time monitoring of policy uncertainty and performance, triggering adaptation cycles when divergence thresholds are breached.[23]

Table 2: Mitigating MARL Challenges in High-Frequency Finance

Challenge	**Impact on System Performance**	**Solution/Mitigation Strategy**	**Relevant Framework/Concept**
Non-Stationarity (Concept Drift)	Trained policies degrade quickly when market regimes shift.	Continuous Adaptation via Ensemble of Experts and online policy updating.[23]	Meta-Learning, Online Selection Algorithms
Scalability/Action Space	Joint action space grows exponentially, impeding efficient learning in large collectives.	Hierarchical MARL, Centralized Training/Decentralized Execution (CTDE).[1]	Proximal Policy Optimization (PPO), Stable-Baselines3 [1]
Partial	Agents cannot see the internal states	Learning representations of	Bayesian Inference, Belief State

Observability	or policies of rival agents or the entire market depth.	opponent's policies; using LLMs (Alpha Agents) to infer latent states.[24]	Modeling
Latency Discrepancy	Strategy performance diverges significantly between backtest and live trading.	Latency-aware backtesting (simulating feed/order delays, queue position).[22]	HftBacktest, Co-location Infrastructure [21]

6.2 Risk Management Integration and Adversarial Training

Risk management must be formalized and integrated directly into the learning process, rather than being treated as a secondary filtering mechanism.[23] For Rbotzilla, this is achieved during the reinforcement learning phase. Specialized bottom-layer trading agents, such as $B^{\star}$ [1], are trained to output a three-dimensional action tuple, where the parameter $\omega \in $ controls the explicit trade-off between self-interested profitability and the suppression of risk.[1] This mathematical constraint allows the Risk Agent to dynamically modulate the policy's risk appetite in real-time based on systemic conditions.

To ensure deep resilience, Rbotzilla must utilize the three-stage training process involving an Adversary agent.[1] By pretraining the Adversary to perturb the market with volatility changes, the trading agents are forced to learn policies that are robust under unexpected exogenous shocks. This rigorous training approach is validated by published results demonstrating the superiority of orchestrated financial agent frameworks, such as achieving a 20.42% return and a Sharpe ratio of 2.63 in stock trading tests, significantly outperforming benchmark indices.[7]

VII. Validation and System Metrics

The success of Rbotzilla hinges on a robust validation methodology that closes the gap between simulated performance and live execution, known as the backtesting-live trading divergence.

7.1 Requirement for Latency-Aware Backtesting

Traditional backtesting, which assumes zero or simulated latency, fundamentally misrepresents real-world trading conditions, where network delays, API speed, and execution time introduce significant latency and slippage.[21] To obtain meaningful metrics, Rbotzilla must employ specialized high-frequency trading backtesting frameworks (e.g., HftBacktest)

designed for market replay based on full order book data (L2 Market-By-Price and L3 Market-By-Order feeds).[22]

The validation framework must accurately simulate three critical parameters: **feed latency** (the time delay in receiving market data), **order latency** (the time delay in sending the order to the exchange), and the precise **order queue position** at the exchange, which determines the probability and speed of order fill.[22] This meticulous simulation ensures that the policies derived from training are executable and profitable within the constrained, low-latency infrastructure required for deployment. Furthermore, many modern backtesting frameworks provide the ability to deploy the exact same algorithm code directly as a live trading bot, ensuring maximum fidelity between the tested policy and the deployed Execution Agent.[26]

7.2 Comprehensive Performance Evaluation Metrics

Due to the complexity and mission-critical nature of the multi-agent system, evaluation metrics must be multi-layered, providing comprehensive visibility into systemic health.[1]

1. **Agent-Level Performance Metrics:** These capture the behavior and success of individual components, including agent profitability, risk exposure (e.g., Value at Risk, Maximum Loss), market participation rate, and specific behaviors like quoting style.[1]
2. **Market-Level Outcomes:** Standard quantitative measures of the portfolio's aggregated performance, such as Return, Sharpe Ratio, and Maximum Drawdown (MDD).[7]
3. **Multi-Agent Interaction Metrics:** These assess the operational stability of the hive mind, measuring communication efficiency (A2A messages per second), conflict resolution rates by the Orchestrator, and policy divergence among collaborative agents.[1]

This quantitative transparency is essential for continuous optimization and provides the necessary empirical data to compare the performance of specialized agents in heterogeneous competitive settings.[1] Given the significant capital expenditure associated with low-latency infrastructure, continuous, rigorous metric tracking is required to justify the operational expenditures and ensure the Rbotzilla system maintains a statistically significant alpha-generating edge.

VIII. Conclusion

The construction of Rbotzilla as a multiple AI "hive mind" demands a fusion of cutting-edge Multi-Agent Reinforcement Learning (MARL) architecture with specialized, low-latency infrastructure. The key conclusion is that the system's resilience and profitability rely entirely on effective coordination and robust policy adaptation.

The centralized Orchestrator, or "Borg Queen," is not optional; it is the necessary guarantor of systemic stability, mitigating error propagation and resolving conflicts arising from decentralized decision-making.[5] Integration of LLMs, such as those derived from OpenAI, must be achieved using finance-specialized models (e.g., FinGPT derivatives) and decoupled

from the execution pipeline using asynchronous, event-driven architectures to manage inference latency.[13] The system must utilize techniques like Confidence-Weighted Scoring and dynamic correlation tracking to transform probabilistic LLM outputs into reliable, risk-adjusted trading signals.[9]

Finally, sustained performance requires rigorous MLOps—utilizing checkpoints (like the observed frozen2snap.zip) for transfer learning and disaster recovery [20]—and an unwavering commitment to realism in validation. All policies must be tested in latency-aware backtesting environments that simulate feed, order, and queue latency, ensuring that strategies successful in simulation translate reliably into profitable, low-latency live execution.[26] The resulting architecture is a complex, hierarchical system designed not for static optimization, but for continuous, adversarial adaptation to the non-stationary competitive environment of the financial markets.

Works cited

1. Multi-Agent Reinforcement Learning for Market Making: Competition without Collusion, accessed December 9, 2025, https://arxiv.org/html/2510.25929v1
2. Multi-agent Systems in Finance: Enhancing Decision-Making and Market Analysis, accessed December 9, 2025, https://smythos.com/developers/agent-development/multi-agent-systems-in-finance/
3. AIGlobalStabilityIntiative/The-Borg-Hive-mind-AI-Collective ... - GitHub, accessed December 9, 2025, https://github.com/AIGlobalStabilityIntiative/The-Borg-Hive-mind-AI-Collective-System
4. What is low latency algorithmic trading? - Aerospike, accessed December 9, 2025, https://aerospike.com/glossary/low-latency-algorithmic-trading/
5. Overcoming Challenges in Multi-Agent Large Language Models: Strategies for Success | by Gelareh Taghizadeh | Medium, accessed December 9, 2025, https://medium.com/@gelareh.taghizadeh_63525/navigating-through-pitfalls-in-multi-agent-llms-1e9d5c2b6d0f
6. Orchestration Framework for Financial Agents: From Algorithmic Trading to Agentic Trading - arXiv, accessed December 9, 2025, https://arxiv.org/html/2512.02227v1
7. Orchestration Framework for Financial Agents: From Algorithmic Trading to Agentic Trading - arXiv, accessed December 9, 2025, https://arxiv.org/pdf/2512.02227
8. What is AI Agent Communication? - IBM, accessed December 9, 2025, https://www.ibm.com/think/topics/ai-agent-communication
9. ashioyajotham/fingpt_trader: A quant trading system platform based on FinGPT, demonstrating new applications of large pre-trained Language Models in quantitative finance. - GitHub, accessed December 9, 2025, https://github.com/ashioyajotham/fingpt_trader

10. Integrating Large Language Models and Reinforcement Learning for Sentiment-Driven Quantitative Trading - arXiv, accessed December 9, 2025, https://arxiv.org/html/2510.10526v1
11. Large Language Models in equity markets: applications, techniques, and insights - PMC, accessed December 9, 2025, https://pmc.ncbi.nlm.nih.gov/articles/PMC12421730/
12. Large Language Models in equity markets: applications, techniques, and insights - Frontiers, accessed December 9, 2025, https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2025.1608365/full
13. An End-To-End LLM Enhanced Trading System - arXiv, accessed December 9, 2025, https://arxiv.org/html/2502.01574v1
14. FinGPT: Open-Source Financial Large Language Models! Revolutionize We release the trained model on HuggingFace. - GitHub, accessed December 9, 2025, https://github.com/AI4Finance-Foundation/FinGPT
15. Best 5 Frameworks To Build Multi-Agent AI Applications - GetStream.io, accessed December 9, 2025, https://getstream.io/blog/multiagent-ai-frameworks/
16. Crew AI, accessed December 9, 2025, https://www.crewai.com/
17. AutoGen Tutorial: A Guide to Building AI Agents - Codecademy, accessed December 9, 2025, https://www.codecademy.com/article/autogen-tutorial-build-ai-agents
18. AutoGen Tutorial: Build Multi-Agent AI Applications - DataCamp, accessed December 9, 2025, https://www.datacamp.com/tutorial/autogen-tutorial
19. A Multi-Agent Framework for Quantitative Finance : An Application to Portfolio Management Analytics - ACL Anthology, accessed December 9, 2025, https://aclanthology.org/2025.emnlp-industry.55.pdf
20. AI Checkpoints: How They Work and More - WEKA, accessed December 9, 2025, https://www.weka.io/learn/glossary/ai-ml/ai-checkpoints/
21. Latency in Trading: Why Every Millisecond Matters - Lares Algotech, accessed December 9, 2025, https://laresalgotech.com/latency-in-trading-why-every-millisecond-matters/
22. HftBacktest — hftbacktest, accessed December 9, 2025, https://hftbacktest.readthedocs.io/
23. Continuous Adaptation in Non-Stationary Reinforcement Learning | by Ursina Sanderink, accessed December 9, 2025, https://medium.com/@sanderink.ursina/continuous-adaptation-in-non-stationary-reinforcement-learning-0860f33e1e73
24. Multi-Agent Reinforcement Learning: A Review of Challenges and Applications - MDPI, accessed December 9, 2025, https://www.mdpi.com/2076-3417/11/11/4948
25. [1906.04737] Dealing with Non-Stationarity in Multi-Agent Deep Reinforcement Learning - arXiv, accessed December 9, 2025, https://arxiv.org/abs/1906.04737
26. High-Frequency Trading Backtesting Tool - HftBacktest - Read the Docs, accessed December 9, 2025, https://hftbacktest.readthedocs.io/en/py-v2.1.0/index.html